

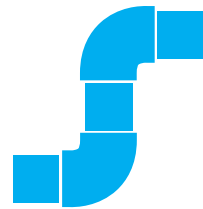
In fact, multiply can go even faster than six add times because of the use of *carry save adders* (see [Section A.6 in Appendix A](#)), and because it is easy to **pipeline** such a design to be able to support many multiplies simultaneously (see [Chapter 4](#)).

Multiply in LEGv8

To produce a properly signed or unsigned 128-bit product, LEGv8 has three instructions: *multiply* (MUL), *signed multiply high* (SMULH) and *unsigned multiply high* (UMULH). To get the integer 64-bit product, the programmer use MUL. To get the upper 64 bits of the 128-bit product, the programmer uses either SMULH or UMULH, depending on the types of multiplier and multiplicand.

Summary

Multiplication hardware simply shifts and adds, as derived from the paper-and-pencil method learned in grammar school. Compilers even use shift instructions for multiplications by powers of 2. With much more hardware we can do the adds in **parallel**, and do them much faster.



PIPELINING



PARALLELISM

LEGv8 multiply instructions do not set the overflow condition code, so it is up to the software to check to see if the product is too big to fit in 64 bits. There is no overflow if the upper 64 bits is 0 for UMULH or the replicated sign of the lower 64 bits for SMULH.

Hardware/Software Interface

3.4 Division

The reciprocal operation of multiply is divide, an operation that is even less frequent and even quirkier. It even offers the opportunity to perform a mathematically invalid operation: dividing by 0.

Let's start with an example of long division using decimal numbers to recall the names of the operands and the division algorithm from grammar school. For reasons similar to those in the previous section, we limit the decimal digits to just 0 or 1. The example is dividing $1,001,010_{\text{ten}}$ by 1000_{ten} :

$$\begin{array}{r}
 \begin{array}{r}
 \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \\
 \underline{-1000} \\
 10 \\
 \underline{101} \\
 1010 \\
 \underline{-1000} \\
 10_{\text{ten}}
 \end{array}
 \end{array}$$

Quotient
Dividend
Remainder

Divide et impera.

Latin for “Divide and rule,” ancient political maxim cited by Machiavelli, 1532

- dividend** A number being divided.
- divisor** A number that the dividend is divided by.
- quotient** The primary result of a division; a number that when multiplied by the divisor and added to the remainder produces the dividend.
- remainder** The secondary result of a division; a number that when added to the product of the quotient and the divisor produces the dividend.

Divide’s two operands, called the **dividend** and **divisor**, and the result, called the **quotient**, are accompanied by a second result, called the **remainder**. Here is another way to express the relationship between the components:

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

where the remainder is smaller than the divisor. Infrequently, programs use the divide instruction just to get the remainder, ignoring the quotient.

The basic division algorithm from grammar school tries to see how big a number can be subtracted, creating a digit of the quotient on each attempt. Our carefully selected decimal example uses just the numbers 0 and 1, so it’s easy to figure out how many times the divisor goes into the portion of the dividend: it’s either 0 times or 1 time. Binary numbers contain only 0 or 1, so binary division is restricted to these two choices, thereby simplifying binary division.

Let’s assume that both the dividend and the divisor are positive and hence the quotient and the remainder are nonnegative. The division operands and both results are 64-bit values, and we will ignore the sign for now.

A Division Algorithm and Hardware

Figure 3.8 shows hardware to mimic our grammar school algorithm. We start with the 64-bit Quotient register set to 0. Each iteration of the algorithm needs to move the divisor to the right one digit, so we start with the divisor placed in the left half of the 128-bit Divisor register and shift it right 1 bit each step to align it with the dividend. The Remainder register is initialized with the dividend.

Figure 3.9 shows three steps of the first division algorithm. Unlike a human, the computer isn’t smart enough to know in advance whether the divisor is smaller

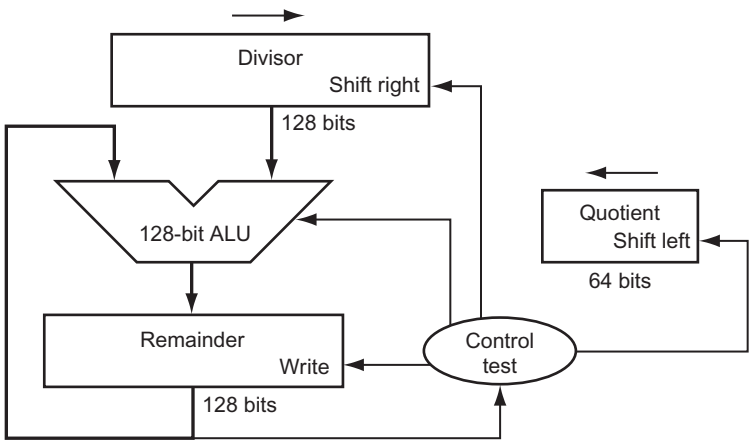


FIGURE 3.8 First version of the division hardware. The Divisor register, ALU, and Remainder register are all 128 bits wide, with only the Quotient register being 62 bits. The 64-bit divisor starts in the left half of the Divisor register and is shifted right 1 bit each iteration. The remainder is initialized with the dividend. Control decides when to shift the Divisor and Quotient registers and when to write the new value into the Remainder register.

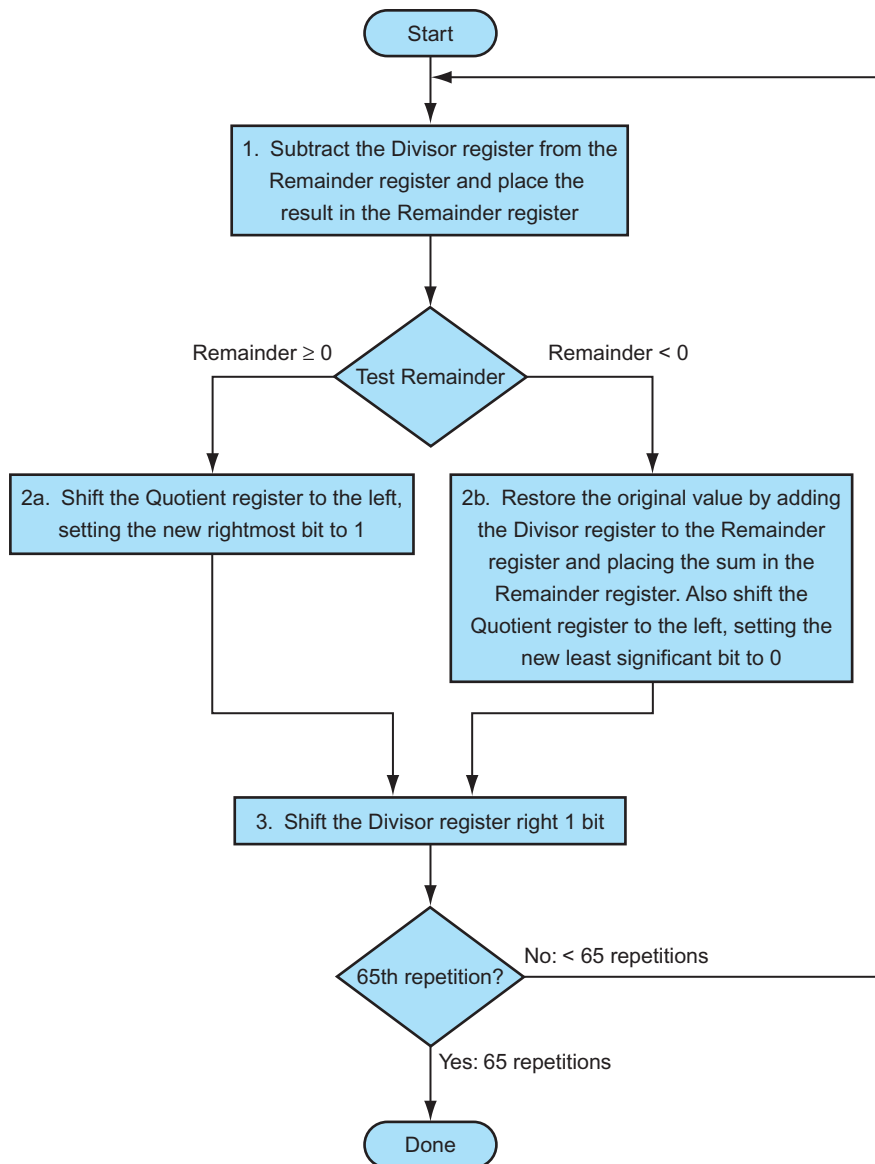


FIGURE 3.9 A division algorithm, using the hardware in Figure 3.8. If the remainder is positive, the divisor did go into the dividend, so step 2a generates a 1 in the quotient. A negative remainder after step 1 means that the divisor did not go into the dividend, so step 2b generates a 0 in the quotient and adds the divisor to the remainder, thereby reversing the subtraction of step 1. The final shift, in step 3, aligns the divisor properly, relative to the dividend for the next iteration. These steps are repeated 65 times.

than the dividend. It must first subtract the divisor in step 1; remember that this is how we performed comparison. If the result is positive, the divisor was smaller or equal to the dividend, so we generate a 1 in the quotient (step 2a). If the result is negative, the next step is to restore the original value by adding the divisor back to the remainder and generate a 0 in the quotient (step 2b). The divisor is shifted right, and then we iterate again. The remainder and quotient will be found in their namesake registers after the iterations complete.

EXAMPLE

ANSWER

A Divide Algorithm

Using a 4-bit version of the algorithm to save pages, let's try dividing 7_{ten} by 2_{ten} , or $0000\ 0111_{\text{two}}$ by 0010_{two} .

Figure 3.10 shows the value of each register for each of the steps, with the quotient being 3_{ten} and the remainder 1_{ten} . Notice that the test in step 2 of whether the remainder is positive or negative simply checks whether the sign bit of the Remainder register is a 0 or 1. The surprising requirement of this algorithm is that it takes $n + 1$ steps to get the proper quotient and remainder.

This algorithm and hardware can be refined to be faster and cheaper. The speed-up comes from shifting the operands and the quotient simultaneously with the

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	⓪110 0111
	2b: Rem < 0 ⇒ +Div, LSL Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	⓪111 0111
	2b: Rem < 0 ⇒ +Div, LSL Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	⓪111 1111
	2b: Rem < 0 ⇒ +Div, LSL Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	⓪000 0011
	2a: Rem ≥ 0 ⇒ LSL Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	⓪000 0001
	2a: Rem ≥ 0 ⇒ LSL Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

FIGURE 3.10 Division example using the algorithm in Figure 3.9. The bit examined to determine the next step is circled in color.

subtraction. This refinement halves the width of the adder and registers by noticing where there are unused portions of registers and adders. Figure 3.11 shows the revised hardware.

Signed Division

So far, we have ignored signed numbers in division. The simplest solution is to remember the signs of the divisor and dividend and then negate the quotient if the signs disagree.

Elaboration: The one complication of signed division is that we must also set the sign of the remainder. Remember that the following equation must always hold:

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

To understand how to set the sign of the remainder, let's look at the example of dividing all the combinations of $\pm 7_{\text{ten}}$ by $\pm 2_{\text{ten}}$. The first case is easy:

$$+7 \div +2: \text{Quotient} = +3, \text{Remainder} = +1$$

Checking the results:

$$+7 = 3 \times 2 + (+1) = 6 + 1$$

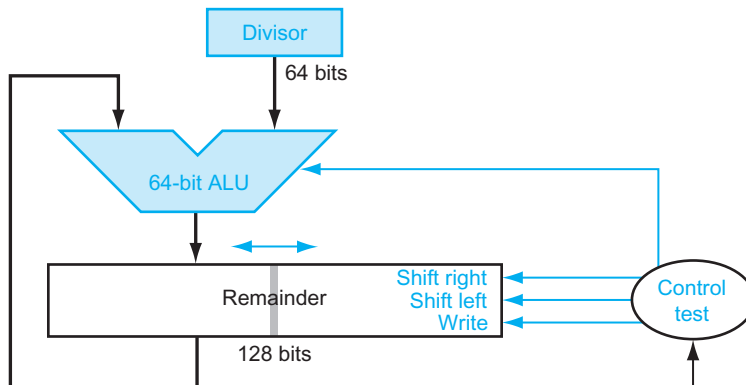


FIGURE 3.11 An improved version of the division hardware. The Divisor register, ALU, and Quotient register are all 64 bits wide, with only the Remainder register left at 128 bits. Compared to Figure 3.8, the ALU and Divisor registers are halved and the remainder is shifted left. This version also combines the Quotient register with the right half of the Remainder register. (As in Figure 3.5, the Remainder register should really be 129 bits to make sure the carry out of the adder is not lost.)

If we change the sign of the dividend, the quotient must change as well:

$$-7 \div +2: \text{Quotient} = -3$$

Rewriting our basic formula to calculate the remainder:

$$\begin{aligned} \text{Remainder} &= (\text{Dividend} - \text{Quotient} \times \text{Divisor}) = -7 - (-3 \times 2) \\ &= -7 - (-6) = -1 \end{aligned}$$

So,

$$-7 \div +2: \text{Quotient} = -3, \text{Remainder} = -1$$

Checking the results again:

$$-7 = -3 \times 2 + (-1) = -6 - 1$$

The reason the answer isn't a quotient of -4 and a remainder of $+1$, which would also fit this formula, is that the absolute value of the quotient would then change depending on the sign of the dividend and the divisor! Clearly, if

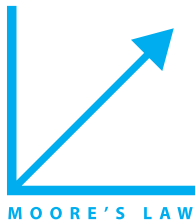
$$-(x \div y) \neq (-x) \div y$$

programming would be an even greater challenge. This anomalous behavior is avoided by following the rule that the dividend and remainder must have identical signs, no matter what the signs of the divisor and quotient.

We calculate the other combinations by following the same rule:

$$\begin{aligned} +7 \div -2: \text{Quotient} &= -3, \text{Remainder} = +1 \\ -7 \div -2: \text{Quotient} &= +3, \text{Remainder} = -1 \end{aligned}$$

Thus, the correctly signed division algorithm negates the quotient if the signs of the operands are opposite and makes the sign of the nonzero remainder match the dividend.

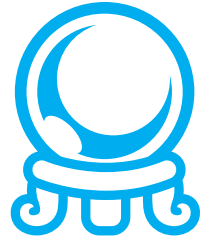


Faster Division

Moore's Law applies to division hardware as well as multiplication, so we would like to be able to speed up division by throwing hardware at it. We used many adders to speed up multiply, but we cannot do the same trick for divide. The reason is that we need to know the sign of the difference before we can perform the next step of the algorithm, whereas with multiply we could calculate the 64 partial products immediately.

There are techniques to produce more than one bit of the quotient per step. The *SRT division* technique tries to **predict** several quotient bits per step, using a table lookup based on the upper bits of the dividend and remainder. It relies on subsequent steps to correct wrong predictions. A typical value today is 4 bits. The key is guessing the value to subtract. With binary division, there is only a single choice. These algorithms use 6 bits from the remainder and 4 bits from the divisor to index a table that determines the guess for each step.

The accuracy of this fast method depends on having proper values in the lookup table. The *Fallacy* on page 244 in Section 3.9 shows what can happen if the table is incorrect.



P R E D I C T I O N

Divide in LEGv8

You may have already observed that the same sequential hardware can be used for both multiply and divide in Figures 3.5 and 3.11. The only requirement is a 128-bit register that can shift left or right and a 64-bit ALU that adds or subtracts.

To handle both signed integers and unsigned integers, LEGv8 has two instructions: *signed divide* (SDIV) and *divide unsigned* (UDIV).

Summary

The common hardware support for multiply and divide allows LEGv8 to provide a single pair of 64-bit registers that are used both for multiply and divide. We accelerate division by predicting multiple quotient bits and then correcting mispredictions later. Figure 3.12 summarizes the enhancements to the LEGv8 architecture for the last two sections.

LEGv8 divide instructions ignore overflow, so software must determine whether the quotient is too large. In addition to overflow, division can also result in an improper calculation: division by 0. Some computers distinguish these two anomalous events. LEGv8 software must check the divisor to discover division by 0 as well as overflow.

**Hardware/
Software
Interface**

Elaboration: An even faster algorithm does not immediately add the divisor back if the remainder is negative. It simply *adds* the dividend to the shifted remainder in the following step, since $(r + d) \times 2 - d = r - 2 + d \times 2 - d = r \times 2 + d$. This *nonrestoring* division algorithm, which takes one clock cycle per step, is explored further in the exercises; the algorithm above is called *restoring* division. A third algorithm that doesn't save the result of the subtract if it's negative is called a *nonperforming* division algorithm. It averages one-third fewer arithmetic operations.

LEGv8 assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	ADD X1, X2, X3	$X1 = X2 + X3$	Three register operands
	subtract	SUB X1, X2, X3	$X1 = X2 - X3$	Three register operands
	add immediate	ADDI X1, X2, 20	$X1 = X2 + 20$	Used to add constants
	subtract immediate	SUBI X1, X2, 20	$X1 = X2 - 20$	Used to subtract constants
	add and set flags	ADDS X1, X2, X3	$X1 = X2 + X3$	Add, set condition codes
	subtract and set flags	SUBS X1, X2, X3	$X1 = X2 - X3$	Subtract, set condition codes
	add immediate and set flags	ADDIS X1, X2, 20	$X1 = X2 + 20$	Add constant, set condition codes
	subtract immediate and set flags	SUBIS X1, X2, 20	$X1 = X2 - 20$	Subtract constant, set condition codes
	multiply	MUL X1, X2, X3	$X1 = X2 \times X3$	Lower 64-bits of 128-bit product
	signed multiply high	SMULH X1, X2, X3	$X1 = X2 \times X3$	Upper 64-bits of 128-bit signed product
	unsigned multiply high	UMULH X1, X2, X3	$X1 = X2 \times X3$	Upper 64-bits of 128-bit unsigned product
	signed divide	SDIV X1, X2, X3	$X1 = X2 / X3$	Divide, treating operands as signed
	unsigned divide	UDIV X1, X2, X3	$X1 = X2 / X3$	Divide, treating operands as unsigned
Data transfer	load register	LDUR X1, [X2, 40]	$X1 = \text{Memory}[X2 + 40]$	Doubleword from memory to register
	store register	STUR X1, [X2, 40]	$\text{Memory}[X2 + 40] = X1$	Doubleword from register to memory
	load signed word	LDURSW X1, [X2, 40]	$X1 = \text{Memory}[X2 + 40]$	Word from memory to register
	store word	STURW X1, [X2, 40]	$\text{Memory}[X2 + 40] = X1$	Word from register to memory
	load half	LDURH X1, [X2, 40]	$X1 = \text{Memory}[X2 + 40]$	Halfword memory to register
	store half	STURH X1, [X2, 40]	$\text{Memory}[X2 + 40] = X1$	Halfword register to memory
	load byte	LDURB X1, [X2, 40]	$X1 = \text{Memory}[X2 + 40]$	Byte from memory to register
	store byte	STURB X1, [X2, 40]	$\text{Memory}[X2 + 40] = X1$	Byte from register to memory
	load exclusive register	LDXR X1, [X2, 0]	$X1 = \text{Memory}[X2]$	Load; 1st half of atomic swap
	store exclusive register	STXR X1, X3, [X2]	$\text{Memory}[X2] = X1; X3 = 0 \text{ or } 1$	Store; 2nd half of atomic swap
	move wide with zero	MOVZ X1, 20	$X1 = 20 \text{ or } 20 * 2^{16} \text{ or } 20 * 2^{32} \text{ or } 20 * 2^{48}$	Loads 16-bit constant, rest zeros
	move wide with keep	MOVK X1, 20	$X1 = 20 \text{ or } 20 * 2^{16} \text{ or } 20 * 2^{32} \text{ or } 20 * 2^{48}$	Loads 16-bit constant, rest unchanged
Logical	and	AND X1, X2, X3	$X1 = X2 \& X3$	Three reg. operands; bit-by-bit AND
	inclusive or	ORR X1, X2, X3	$X1 = X2 X3$	Three reg. operands; bit-by-bit OR
	exclusive or	EOR X1, X2, X3	$X1 = X2 \wedge X3$	Three reg. operands; bit-by-bit XOR
	and immediate	ANDI X1, X2, 20	$X1 = X2 \& 20$	Bit-by-bit AND reg with constant
	inclusive or immediate	ORRI X1, X2, 20	$X1 = X2 20$	Bit-by-bit OR reg with constant
	exclusive or immediate	EORI X1, X2, 20	$X1 = X2 \wedge 20$	Bit-by-bit XOR reg with constant
	logical shift left	LSL X1, X2, 10	$X1 = X2 \ll 10$	Shift left by constant
	logical shift right	LSR X1, X2, 10	$X1 = X2 \gg 10$	Shift right by constant
Conditional branch	compare and branch on equal 0	CBZ X1, 25	if ($X1 == 0$) go to PC + 4 + 100	Equal 0 test; PC-relative branch
	compare and branch on not equal 0	CBNZ X1, 25	if ($X1 != 0$) go to PC + 4 + 100	Not equal 0 test; PC-relative
	branch conditionally	B.cond 25	if (condition true) go to PC + 4 + 100	Test condition codes; if true, branch
Unconditional jump	branch	B 2500	go to PC + 4 + 10000	Branch to target address; PC-relative
	branch to register	BR X30	go to X30	For switch, procedure return
	branch with link	BL 2500	$X30 = \text{PC} + 4; \text{PC} + 4 + 10000$	For procedure call PC-relative

FIGURE 3.12 LEGv8 core architecture. LEGv8 machine language is listed in the LEGv8 Reference Data Card at the front of this book.

3.5 Floating Point

Going beyond signed and unsigned integers, programming languages support numbers with fractions, which are called *reals* in mathematics. Here are some examples of reals:

$3.14159265\dots_{\text{ten}}$ (π)

$2.71828\dots_{\text{ten}}$ (e)

0.000000001_{ten} or $1.0_{\text{ten}} \times 10^{-9}$ (seconds in a nanosecond)

$3,155,760,000_{\text{ten}}$ or $3.15576_{\text{ten}} \times 10^9$ (seconds in a typical century)

Notice that in the last case, the number didn't represent a small fraction, but it was bigger than we could represent with a 32-bit signed integer. The alternative notation for the last two numbers is called **scientific notation**, which has a single digit to the left of the decimal point. A number in scientific notation that has no leading 0s is called a **normalized** number, which is the usual way to write it. For example, $1.0_{\text{ten}} \times 10^{-9}$ is in normalized scientific notation, but $0.1_{\text{ten}} \times 10^{-8}$ and $10.0_{\text{ten}} \times 10^{-10}$ are not.

Just as we can show decimal numbers in scientific notation, we can also show binary numbers in scientific notation:

$$1.0_{\text{two}} \times 2^{-1}$$

To keep a binary number in the normalized form, we need a base that we can increase or decrease by exactly the number of bits the number must be shifted to have one nonzero digit to the left of the decimal point. Only a base of 2 fulfills our need. Since the base is not 10, we also need a new name for decimal point; *binary point* will do fine.

Computer arithmetic that supports such numbers is called **floating point** because it represents numbers in which the binary point is not fixed, as it is for integers. The programming language C uses the name *float* for such numbers. Just as in scientific notation, numbers are represented as a single nonzero digit to the left of the binary point. In binary, the form is

$$1.\text{xxxxxxxx}_{\text{two}} \times 2^{\text{yyyy}}$$

(Although the computer represents the exponent in base 2 as well as the rest of the number, to simplify the notation we show the exponent in decimal.)

A standard scientific notation for reals in the normalized form offers three advantages. It simplifies exchange of data that includes floating-point numbers; it simplifies the floating-point arithmetic algorithms to know that numbers will

*Speed gets you
nowhere if you're
headed the wrong way.*

American proverb

scientific notation A notation that renders numbers with a single digit to the left of the decimal point.

normalized A number in floating-point notation that has no leading 0s.

floating point Computer arithmetic that represents numbers in which the binary point is not fixed.

always be in this form; and it increases the accuracy of the numbers that can be stored in a word, since real digits to the right of the binary point replace the unnecessary leading 0s.

Floating-Point Representation

fraction The value, generally between 0 and 1, placed in the fraction field. The fraction is also called the *mantissa*.

exponent In the numerical representation system of floating-point arithmetic, the value that is placed in the exponent field.

A designer of a floating-point representation must find a compromise between the size of the **fraction** and the size of the **exponent**, because a fixed word size means you must take a bit from one to add a bit to the other. This tradeoff is between precision and range: increasing the size of the fraction enhances the precision of the fraction, while increasing the size of the exponent increases the range of numbers that can be represented. As our design guideline from Chapter 2 reminds us, good design demands good compromise.

Floating-point numbers are usually a multiple of the size of a word. The representation of an LEGv8 floating-point number is shown below, where *s* is the sign of the floating-point number (1 meaning negative), *exponent* is the value of the 8-bit exponent field (including the sign of the exponent), and *fraction* is the 23-bit number. As we recall from Chapter 2, this representation is *sign and magnitude*, since the sign is a separate bit from the rest of the number.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

In general, floating-point numbers are of the form

(-1)^S × F × 2^E

F involves the value in the fraction field and E involves the value in the exponent field; the exact relationship to these fields will be spelled out soon. (We will shortly see that LEGv8 does something slightly more sophisticated.)

These chosen sizes of exponent and fraction give LEGv8 computer arithmetic an extraordinary range. Fractions almost as small as 2.0_{ten} × 10⁻³⁸ and numbers almost as large as 2.0_{ten} × 10³⁸ can be represented in a computer. Alas, extraordinary differs from infinite, so it is still possible for numbers to be too large. Thus, overflow interrupts can occur in floating-point arithmetic as well as in integer arithmetic. Notice that **overflow** here means that the exponent is too large to be represented in the exponent field.

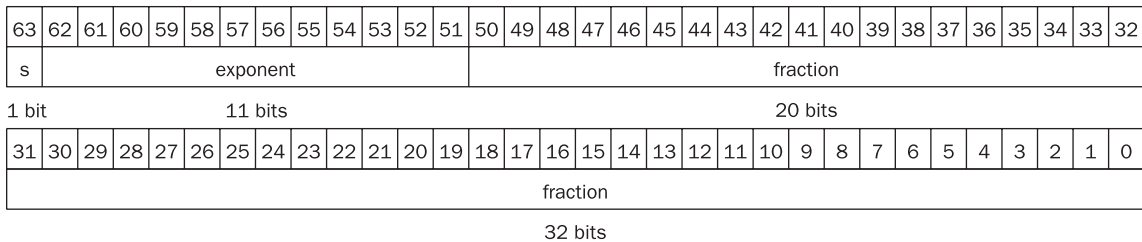
Floating point offers a new kind of exceptional event as well. Just as programmers will want to know when they have calculated a number that is too large to be represented, they will want to know if the nonzero fraction they are calculating has become so small that it cannot be represented; either event could result in a program giving incorrect answers. To distinguish it from overflow, we call this event **underflow**. This situation occurs when the negative exponent is too large to fit in the exponent field.

overflow (floating-point) A situation in which a positive exponent becomes too large to fit in the exponent field.

underflow (floating-point) A situation in which a negative exponent becomes too large to fit in the exponent field.

One way to reduce the chances of underflow or overflow is to offer another format that has a larger exponent. In C, this number is called *double*, and operations on doubles are called **double precision** floating-point arithmetic; **single precision** floating point is the name of the earlier format.

The representation of a double precision floating-point number takes one LEGv8 doubleword, as shown below, where *s* is still the sign of the number, *exponent* is the value of the 11-bit exponent field, and *fraction* is the 52-bit number in the fraction field.



LEGv8 double precision allows numbers almost as small as $2.0_{\text{ten}} \times 10^{-308}$ and almost as large as $2.0_{\text{ten}} \times 10^{308}$. Although double precision does increase the exponent range, its primary advantage is its greater precision because of the much larger fraction.

Exceptions and Interrupts

What should happen on an overflow or underflow to let the user know that a problem occurred? LEGv8 can raise an **exception**, also called an **interrupt** on many computers. An exception or interrupt is essentially an unscheduled procedure call. The address of the instruction that overflowed is saved in a register, and the computer jumps to a predefined address to invoke the appropriate routine for that exception. The interrupted address is saved so that in some situations the program can continue after corrective code is executed. (Section 4.9 covers exceptions in more detail; Chapter 5 describes other situations where exceptions and interrupts occur.)

double precision

A floating-point value represented in a 64-bit doubleword.

single precision

A floating-point value represented in a 32-bit word.

exception Also called **interrupt**. An unscheduled event that disrupts program execution; used to detect overflow.

interrupt An exception that comes from outside of the processor. (Some architectures use the term *interrupt* for all exceptions.)

IEEE 754 Floating-Point Standard

These formats go beyond LEGv8. They are part of the *IEEE 754 floating-point standard*, found in virtually every computer invented since 1980. This standard has greatly improved both the ease of porting floating-point programs and the quality of computer arithmetic.

To pack even more bits into the number, IEEE 754 makes the leading 1 bit of normalized binary numbers implicit. Hence, the number is actually 24 bits long in single precision (implied 1 and a 23-bit fraction), and 53 bits long in double precision (1 + 52). To be precise, we use the term *significand* to represent the 24- or 53-bit number that is 1 plus the fraction, and *fraction* when we mean the 23- or

52-bit number. Since 0 has no leading 1, it is given the reserved exponent value 0 so that the hardware won't attach a leading 1 to it.

Thus $00 \dots 00_{\text{two}}$ represents 0; the representation of the rest of the numbers uses the form from before with the hidden 1 added:

$$(-1)^S \times (1 + \text{Fraction}) \times 2^E$$

where the bits of the fraction represent a number between 0 and 1 and E specifies the value in the exponent field, to be given in detail shortly. If we number the bits of the fraction from *left to right* s1, s2, s3, ..., then the value is

$$(-1)^S \times (1 + (s1 \times 2^{-1}) + (s2 \times 2^{-2}) + (s3 \times 2^{-3}) + (s4 \times 2^{-4}) + \dots) \times 2^E$$

Figure 3.13 shows the encodings of IEEE 754 floating-point numbers. Other features of IEEE 754 are special symbols to represent unusual events. For example, instead of interrupting on a divide by 0, software can set the result to a bit pattern representing $+\infty$ or $-\infty$; the largest exponent is reserved for these special symbols. When the programmer prints the results, the program will output an infinity symbol. (For the mathematically trained, the purpose of infinity is to form topological closure of the reals.)

IEEE 754 even has a symbol for the result of invalid operations, such as 0/0 or subtracting infinity from infinity. This symbol is NaN, for *Not a Number*. The purpose of NaNs is to allow programmers to postpone some tests and decisions to a later time in the program when they are convenient.

The designers of IEEE 754 also wanted a floating-point representation that could be easily processed by integer comparisons, especially for sorting. This desire is why the sign is in the most significant bit, allowing a quick test of less than, greater than, or equal to 0. (It's a little more complicated than a simple integer sort, since this notation is essentially sign and magnitude rather than two's complement.)

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

FIGURE 3.13 IEEE 754 encoding of floating-point numbers. A separate sign bit determines the sign. Denormalized numbers are described in the *Elaboration* on page 230. This information is also found in Column 4 of the LEGv8 Reference Data Card at the front of this book.

Placing the exponent before the significand also simplifies the sorting of floating-point numbers using integer comparison instructions, since numbers with bigger exponents look larger than numbers with smaller exponents, as long as both exponents have the same sign.

Negative exponents pose a challenge to simplified sorting. If we use two's complement or any other notation in which negative exponents have a 1 in the most significant bit of the exponent field, a negative exponent will look like a big number. For example, $1.0_{\text{two}} \times 2^{-1}$ would be represented in a single precision as

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
●	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

(Remember that the leading 1 is implicit in the significand.) The value $1.0_{\text{two}} \times 2^{+1}$ would look like the smaller binary number

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The desirable notation must therefore represent the most negative exponent as $00 \dots 00_{\text{two}}$ and the most positive as $11 \dots 11_{\text{two}}$. This convention is called *biased notation*, with the bias being the number subtracted from the normal, unsigned representation to determine the real value.

IEEE 754 uses a bias of 127 for single precision, so an exponent of -1 is represented by the bit pattern of the value $-1 + 127_{\text{ten}}$, or $126_{\text{ten}} = 0111\ 1110_{\text{two}}$, and $+1$ is represented by $1 + 127$, or $128_{\text{ten}} = 1000\ 0000_{\text{two}}$. The exponent bias for double precision is 1023. Biased exponent means that the value represented by a floating-point number is really

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

The range of single precision numbers is then from as small as

$$\pm 1.000000000000000000000000_{\text{two}} \times 2^{-126}$$

to as large as

$$\pm 1.111111111111111111111111_{\text{two}} \times 2^{+127}.$$

Let's demonstrate.

Now let's try going the other direction.

Converting Binary to Decimal Floating Point

EXAMPLE

What decimal number does this single precision float represent?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The sign bit is 1, the exponent field contains 129, and the fraction field contains $1 \times 2^{-2} = 1/4$, or 0.25. Using the basic equation,

ANSWER

$$\begin{aligned}
 (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} &= (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)} \\
 &= -1 \times 1.25 \times 2^2 \\
 &= -1.25 \times 4 \\
 &= -5.0
 \end{aligned}$$

In the next few subsections, we will give the algorithms for floating-point addition and multiplication. At their core, they use the corresponding integer operations on the significands, but extra bookkeeping is necessary to handle the exponents and normalize the result. We first give an intuitive derivation of the algorithms in decimal and then give a more detailed, binary version in the figures.

Elaboration: Following IEEE guidelines, the IEEE 754 committee was reformed 20 years after the standard to see what changes, if any, should be made. The revised standard IEEE 754-2008 includes nearly all the IEEE 754-1985 and adds a 16-bit format (“half precision”) and a 128-bit format (“quadruple precision”). No hardware has yet been built that supports quadruple precision, but it will surely come. The revised standard also adds decimal floating point arithmetic, which IBM mainframes have implemented.

Elaboration: In an attempt to increase range without removing bits from the significand, some computers before the IEEE 754 standard used a base other than 2. For example, the IBM 360 and 370 mainframe computers use base 16. Since changing the IBM exponent by one means shifting the significand by 4 bits, “normalized” base 16 numbers can have up to 3 leading bits of 0s! Hence, hexadecimal digits mean that up to 3 bits must be dropped from the significand, which leads to surprising problems in the accuracy of floating-point arithmetic. IBM mainframes now support IEEE 754 as well as the old hex format.

Floating-Point Addition

Let's add numbers in scientific notation by hand to illustrate the problems in floating-point addition: $9.999_{\text{ten}} \times 10^1 + 1.610_{\text{ten}} \times 10^{-1}$. Assume that we can store only four decimal digits of the significand and two decimal digits of the exponent.

- Step 1. To be able to add these numbers properly, we must align the decimal point of the number that has the smaller exponent. Hence, we need a form of the smaller number, $1.610_{\text{ten}} \times 10^{-1}$, that matches the larger exponent. We obtain this by observing that there are multiple representations of an unnormalized floating-point number in scientific notation:

$$1.610_{\text{ten}} \times 10^{-1} = 0.1610_{\text{ten}} \times 10^0 = 0.01610_{\text{ten}} \times 10^1$$

The number on the right is the version we desire, since its exponent matches the exponent of the larger number, $9.999_{\text{ten}} \times 10^1$. Thus, the first step shifts the significand of the smaller number to the right until its corrected exponent matches that of the larger number. But we can represent only four decimal digits so, after shifting, the number is really

$$0.016 \times 10^1$$

- Step 2. Next comes the addition of the significands:

$$\begin{array}{r} 9.999_{\text{ten}} \\ + \quad 0.016_{\text{ten}} \\ \hline 10.015_{\text{ten}} \end{array}$$

The sum is $10.015_{\text{ten}} \times 10^1$.

- Step 3. This sum is not in normalized scientific notation, so we need to adjust it:

$$10.015_{\text{ten}} \times 10^1 = 1.0015_{\text{ten}} \times 10^2$$

Thus, after the addition we may have to shift the sum to put it into normalized form, adjusting the exponent appropriately. This example shows shifting to the right, but if one number were positive and the other were negative, it would be possible for the sum to have many leading 0s, requiring left shifts. Whenever the exponent is increased or decreased, we must check for overflow or underflow—that is, we must make sure that the exponent still fits in its field.

- Step 4. Since we assumed that the significand could be only four digits long (excluding the sign), we must round the number. In our grammar school algorithm, the rules truncate the number if the digit to the right of the desired point is between 0 and 4 and add 1 to the digit if the number to the right is between 5 and 9. The number

$$1.0015_{\text{ten}} \times 10^2$$

is rounded to four digits in the significand to

$$1.002_{\text{ten}} \times 10^2$$

since the fourth digit to the right of the decimal point was between 5 and 9. Notice that if we have bad luck on rounding, such as adding 1 to a string of 9s, the sum may no longer be normalized and we would need to perform step 3 again.

Figure 3.14 shows the algorithm for binary floating-point addition that follows this decimal example. Steps 1 and 2 are similar to the example just discussed: adjust the significand of the number with the smaller exponent and then add the two significands. Step 3 normalizes the results, forcing a check for overflow or underflow. The test for overflow and underflow in step 3 depends on the precision of the operands. Recall that the pattern of all 0 bits in the exponent is reserved and used for the floating-point representation of zero. Moreover, the pattern of all 1 bits in the exponent is reserved for indicating values and situations outside the scope of normal floating-point numbers (see the *Elaboration* on page 230). For the example below, remember that for single precision, the maximum exponent is 127, and the minimum exponent is -126.

Binary Floating-Point Addition

Try adding the numbers 0.5_{ten} and -0.4375_{ten} in binary using the algorithm in Figure 3.14.

EXAMPLE

Let's first look at the binary version of the two numbers in normalized scientific notation, assuming that we keep 4 bits of precision:

ANSWER

$$\begin{aligned} 0.5_{\text{ten}} &= 1/2_{\text{ten}} = 1/2_{\text{two}}^1 \\ &= 0.1_{\text{two}} = 0.1_{\text{two}} \times 2^0 = 1.000_{\text{two}} \times 2^{-1} \\ -0.4375_{\text{ten}} &= -7/16_{\text{ten}} = -7/2_{\text{ten}}^4 \\ &= -0.0111_{\text{two}} = -0.0111_{\text{two}} \times 2^0 = -1.110_{\text{two}} \times 2^{-2} \end{aligned}$$

Now we follow the algorithm:

Step 1. The significand of the number with the lesser exponent ($-1.11_{\text{two}} \times 2^{-2}$) is shifted right until its exponent matches the larger number:

$$-1.110_{\text{two}} \times 2^{-2} = -0.111_{\text{two}} \times 2^{-1}$$

Step 2. Add the significands:

$$1.000_{\text{two}} \times 2^{-1} + (-0.111_{\text{two}} \times 2^{-1}) = 0.001_{\text{two}} \times 2^{-1}$$

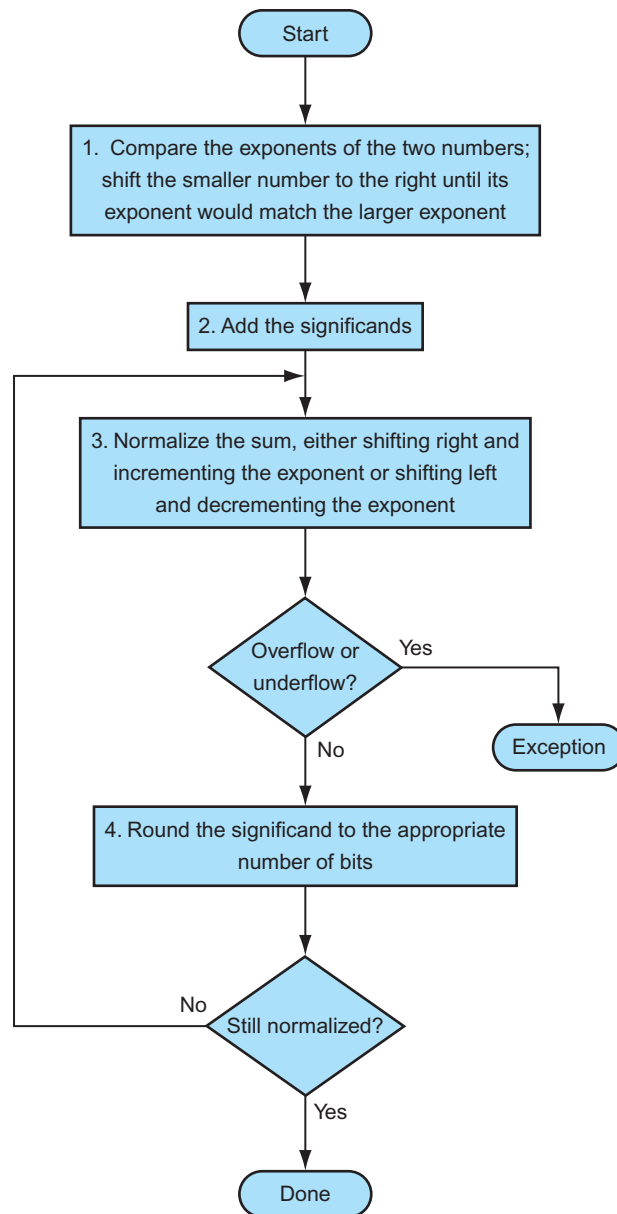


FIGURE 3.14 Floating-point addition. The normal path is to execute steps 3 and 4 once, but if rounding causes the sum to be unnormalized, we must repeat step 3.

Step 3. Normalize the sum, checking for overflow or underflow:

$$\begin{aligned} 0.001_{\text{two}} \times 2^{-1} &= 0.010_{\text{two}} \times 2^{-2} = 0.100_{\text{two}} \times 2^{-3} \\ &= 1.000_{\text{two}} \times 2^{-4} \end{aligned}$$

Since $127 \geq -4 \geq -126$, there is no overflow or underflow. (The biased exponent would be $-4 + 127$, or 123, which is between 1 and 254, the smallest and largest unreserved biased exponents.)

Step 4. Round the sum:

$$1.000_{\text{two}} \times 2^{-4}$$

The sum already fits exactly in 4 bits, so there is no change to the bits due to rounding.

This sum is then

$$\begin{aligned} 1.000_{\text{two}} \times 2^{-4} &= 0.0001000_{\text{two}} = 0.0001_{\text{two}} \\ &= 1/2^4_{\text{ten}} = 1/16_{\text{ten}} = 0.0625_{\text{ten}} \end{aligned}$$

This sum is what we would expect from adding 0.5_{ten} to -0.4375_{ten} .

Many computers dedicate hardware to run floating-point operations as fast as possible. [Figure 3.15](#) sketches the basic organization of hardware for floating-point addition.

Floating-Point Multiplication

Now that we have explained floating-point addition, let's try floating-point multiplication. We start by multiplying decimal numbers in scientific notation by hand: $1.110_{\text{ten}} \times 10^{10} \times 9.200_{\text{ten}} \times 10^{-5}$. Assume that we can store only four digits of the significand and two digits of the exponent.

Step 1. Unlike addition, we calculate the exponent of the product by simply adding the exponents of the operands together:

$$\text{New exponent} = 10 + (-5) = 5$$

Let's do this with the biased exponents as well to make sure we obtain the same result: $10 + 127 = 137$, and $-5 + 127 = 122$, so

$$\text{New exponent} = 137 + 122 = 259$$

This result is too large for the 8-bit exponent field, so something is amiss! The problem is with the bias because we are adding the biases as well as the exponents:

$$\text{New exponent} = (10 + 127) + (-5 + 127) = (5 + 2 \times 127) = 259$$

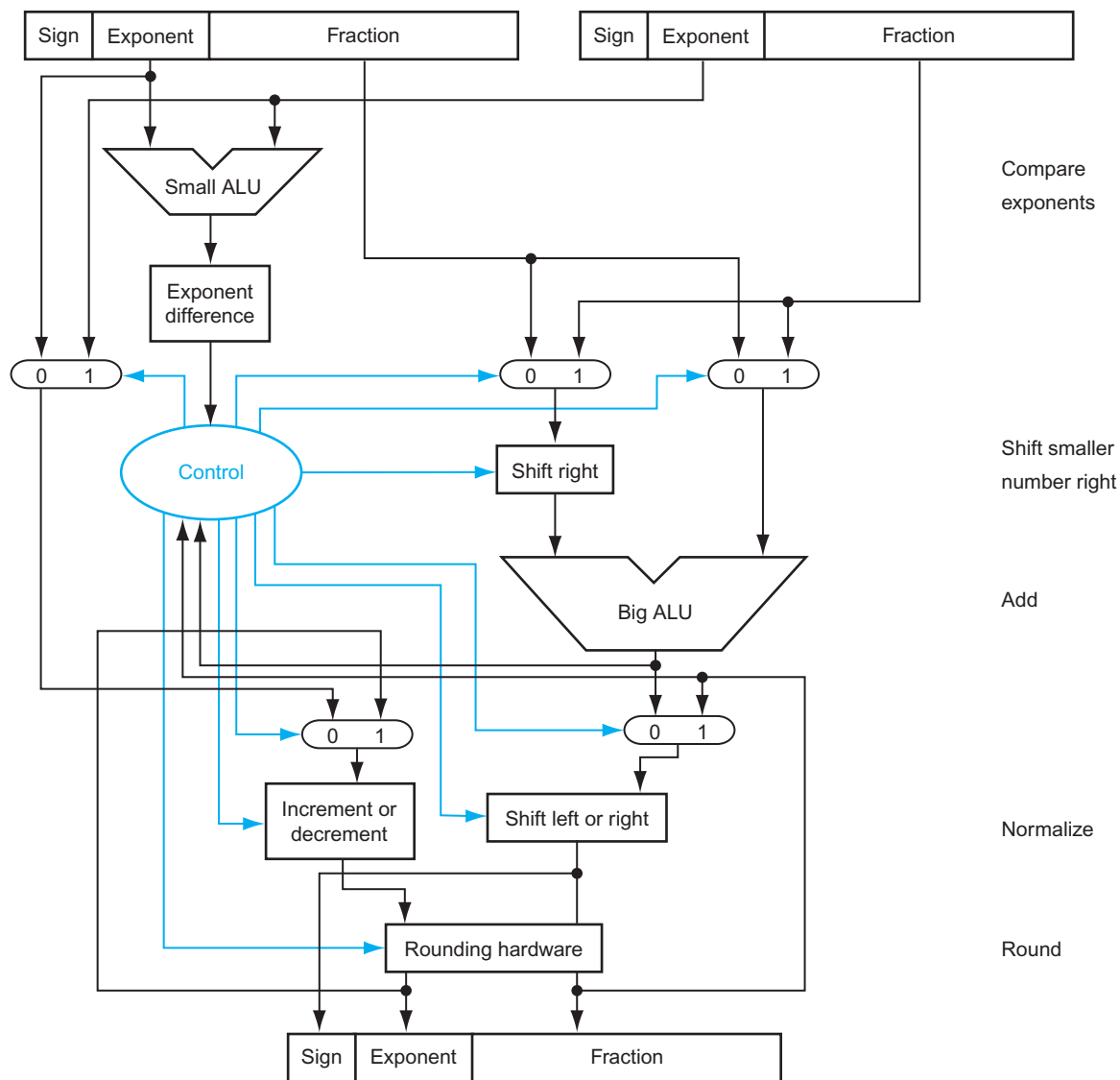


FIGURE 3.15 Block diagram of an arithmetic unit dedicated to floating-point addition. The steps of Figure 3.14 correspond to each block, from top to bottom. First, the exponent of one operand is subtracted from the other using the small ALU to determine which is larger and by how much. This difference controls the three multiplexers; from left to right, they select the larger exponent, the significand of the smaller number, and the significand of the larger number. The smaller significand is shifted right, and then the significands are added together using the big ALU. The normalization step then shifts the sum left or right and increments or decrements the exponent. Rounding then creates the final result, which may require normalizing again to produce the actual final result.

Accordingly, to get the correct biased sum when we add biased numbers, we must subtract the bias from the sum:

$$\text{New exponent} = 137 + 122 - 127 = 259 - 127 = 132 = (5 + 127)$$

and 5 is indeed the exponent we calculated initially.

Step 2. Next comes the multiplication of the significands:

$$\begin{array}{r} 1.110_{\text{ten}} \\ \times 9.200_{\text{ten}} \\ \hline 0000 \\ 0000 \\ 2220 \\ 9990 \\ \hline 1110000_{\text{ten}} \end{array}$$

There are three digits to the right of the decimal point for each operand, so the decimal point is placed six digits from the right in the product significand:

$$10.212000_{\text{ten}}$$

If we can keep only three digits to the right of the decimal point, the product is 10.212×10^5 .

Step 3. This product is unnormalized, so we need to normalize it:

$$10.212_{\text{ten}} \times 10^5 = 1.0212_{\text{ten}} \times 10^6$$

Thus, after the multiplication, the product can be shifted right one digit to put it in normalized form, adding 1 to the exponent. At this point, we can check for overflow and underflow. Underflow may occur if both operands are small—that is, if both have large negative exponents.

Step 4. We assumed that the significand is only four digits long (excluding the sign), so we must round the number. The number

$$1.0212_{\text{ten}} \times 10^6$$

is rounded to four digits in the significand to

$$1.021_{\text{ten}} \times 10^6$$

Step 5. The sign of the product depends on the signs of the original operands. If they are both the same, the sign is positive; otherwise, it's negative. Hence, the product is

$$+1.021_{\text{ten}} \times 10^6$$

The sign of the sum in the addition algorithm was determined by addition of the significands, but in multiplication, the signs of the operands determine the sign of the product.

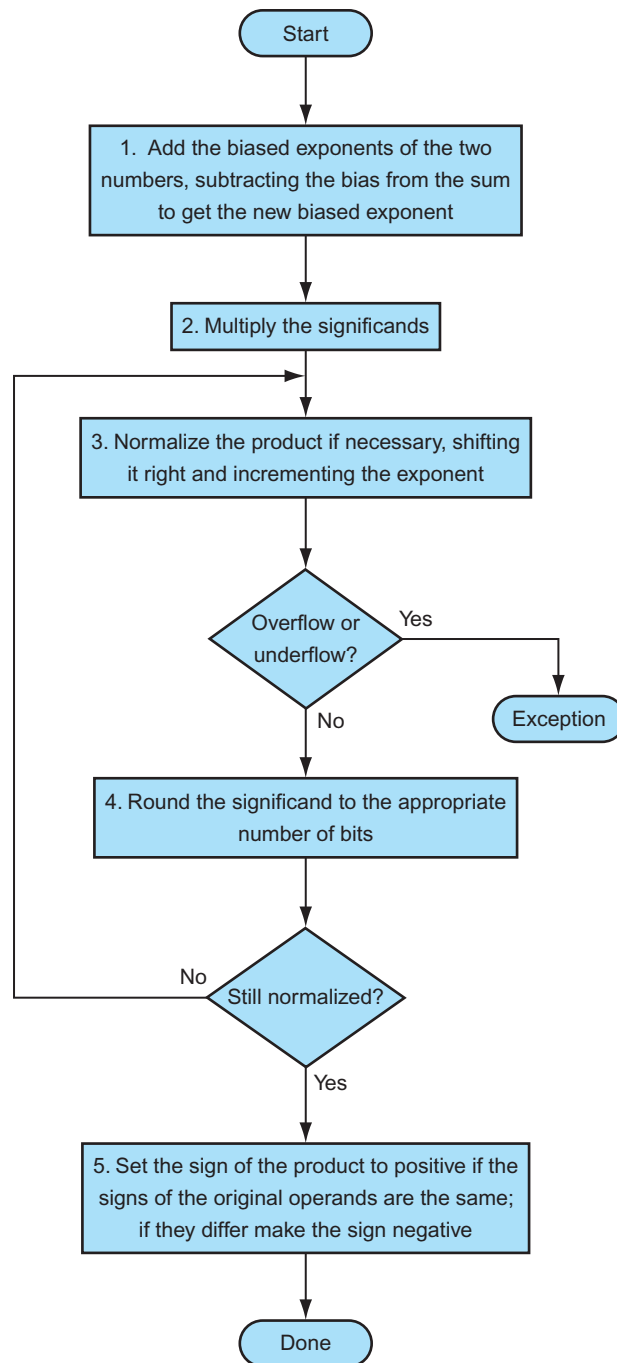


FIGURE 3.16 Floating-point multiplication. The normal path is to execute steps 3 and 4 once, but if rounding causes the sum to be unnormalized, we must repeat step 3.

Once again, as Figure 3.16 shows, multiplication of binary floating-point numbers is quite similar to the steps we have just completed. We start with calculating the new exponent of the product by adding the biased exponents, being sure to subtract one bias to get the proper result. Next is multiplication of significands, followed by an optional normalization step. The size of the exponent is checked for overflow or underflow, and then the product is rounded. If rounding leads to further normalization, we once again check for exponent size. Finally, set the sign bit to 1 if the signs of the operands were different (negative product) or to 0 if they were the same (positive product).

Binary Floating-Point Multiplication

Let's try multiplying the numbers 0.5_{ten} and -0.4375_{ten} , using the steps in Figure 3.16.

In binary, the task is multiplying $1.000_{\text{two}} \times 2^{-1}$ by $-1.110_{\text{two}} \times 2^{-2}$.

Step 1. Adding the exponents without bias:

$$-1 + (-2) = -3$$

or, using the biased representation:

$$\begin{aligned} (-1 + 127) + (-2 + 127) - 127 &= (-1 - 2) + (127 + 127 - 127) \\ &= -3 + 127 = 124 \end{aligned}$$

Step 2. Multiplying the significands:

$$\begin{array}{r} 1.000_{\text{two}} \\ \times 1.110_{\text{two}} \\ \hline 0000 \\ 1000 \\ 1000 \\ 1000 \\ \hline 1110000_{\text{two}} \end{array}$$

The product is $1.110000_{\text{two}} \times 2^{-3}$, but we need to keep it to 4 bits, so it is $1.110_{\text{two}} \times 2^{-3}$.

Step 3. Now we check the product to make sure it is normalized, and then check the exponent for overflow or underflow. The product is already normalized and, since $127 \geq -3 \geq -126$, there is no overflow or underflow. (Using the biased representation, $254 \geq 124 \geq 1$, so the exponent fits.)

EXAMPLE

ANSWER

Step 4. Rounding the product makes no change:

$$1.110_{\text{two}} \times 2^{-3}$$

Step 5. Since the signs of the original operands differ, make the sign of the product negative. Hence, the product is

$$-1.110_{\text{two}} \times 2^{-3}$$

Converting to decimal to check our results:

$$\begin{aligned} -1.110_{\text{two}} \times 2^{-3} &= -0.001110_{\text{two}} = -0.00111_{\text{two}} \\ &= -7/2_{\text{ten}}^5 = -7/32_{\text{ten}} = -0.21875_{\text{ten}} \end{aligned}$$

The product of 0.5_{ten} and -0.4375_{ten} is indeed -0.21875_{ten} .

Floating-Point Instructions in LEGv8

LEGv8 supports the IEEE 754 single-precision and double-precision formats with these instructions:

- Floating-point *addition, single* (FADDS) and *addition, double* (FADD)
- Floating-point *subtraction, single* (FSUBS) and *subtraction, double* (FSUBD)
- Floating-point *multiplication, single* (FMULS) and *multiplication, double* (FMULD)
- Floating-point *division, single* (FDIVS) and *division, double* (FDIVD)
- Floating-point *comparison, single* (FCMPS) and *comparison, double* (FCMPD), with the condition codes given slightly different interpretations

Programmers use `B.cond` to branch based on floating-point comparisons.

The LEGv8 designers decided to add separate floating-point registers. They are called `S0`, `S1`, `S2`, ... for single precision and `D0`, `D1`, `D2`, ... for double precision. Hence, they included separate loads and stores for floating-point registers: `LDURS` and `STURS`. The base registers for floating-point data transfers which are used for addresses remain integer registers. The LEGv8 code to load two single precision numbers from memory, add them, and then store the sum might look like this:

```
LDURS  S4, [X28,c] // Load 32-bit F.P. number into S4
LDURS  S6, [X28,a] // Load 32-bit F.P. number into S6
FADDS  S2, S4, S6  // S2 = S4 + S6 single precision
STURS  S2, [X28,b] // Store 32-bit F.P. number from S2
```

A single precision register is just the lower half of a double-precision register.

Figure 3.17 summarizes the floating-point portion of the LEGv8 architecture revealed in this chapter, with the new pieces to support floating point shown in color.

LEGv8 floating-point operands

Name	Example	Comments
32 floating-point registers	S0, S1, . . . , S31 or D0, D1, . . . , D31	LEGv8 floating-point single precision registers (S0, S1, ..., S31) are just the lower half of the double precision registers (D0, D1, ..., D31)
2 ⁶¹ memory doublewords	Memory[0], Memory[8], . . . , Memory[4,611,686,018,427,387,900]	Accessed only by data transfer instructions. LEGv8 uses byte addresses, so sequential doubleword addresses differ by 8. Memory holds data structures, such as arrays, and spilled registers, such as those saved on procedure calls.

LEGv8 floating-point assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	FP add single	FADDS S2, S4, S6	S2 = S4 + S6	FP add (single precision)
	FP subtract single	FSUBS S2, S4, S6	S2 = S4 - S6	FP sub (single precision)
	FP multiply single	FMULS S2, S4, S6	S2 = S4 × S6	FP multiply (single precision)
	FP divide single	FDIVS S2, S4, S6	S2 = S4 / S6	FP divide (single precision)
	FP add double	FADDD D2, D4, D6	D2 = D4 + D6	FP add (double precision)
	FP subtract double	FSUBD D2, D4, D6	D2 = D4 - D6	FP sub (double precision)
	FP multiply double	FMULD D2, D4, D6	D2 = D4 × D6	FP multiply (double precision)
	FP divide double	FDIVD D2, D4, D6	D2 = D4 / D6	FP divide (double precision)
Conditional branch	FP compare single	FCMPS S4, S6	Test S4 vs. S6	FP compare single precision
	FP compare double	FCMPD D4, D6	Test D4 vs. D6	FP compare double precision
Data transfer	Load single FP	LDURS S1, [X23,100]	S1 = Memory[X23 + 100]	32-bit data to FP register
	Load double FP	LDURD D1, [X23,100]	D1 = Memory[X23 + 100]	64-bit data to FP register
	Store single FP	STURS S1, [X23,100]	Memory[X23 + 100] = S1	32-bit data to memory
	Store double FP	STURD D1, [X23,100]	Memory[X23 + 100] = D1	64-bit data to memory

LEGv8 floating-point machine language

Name	Format	Example					Comments
FADDS	R	241	6	10	4	2	FADDS S2, S4, S6
FSUBS	R	241	6	14	4	2	FSUBS S2, S4, S6
FMULS	R	241	6	2	4	2	FMULS S2, S4, S6
FDIVS	R	241	6	6	4	2	FDIVS S2, S4, S6
FADDD	R	243	6	10	4	2	FADDD D2, D4, D6
FSUBD	R	243	6	14	4	2	FSUBD D2, D4, D6
FMULD	R	243	6	2	4	2	FMULD D2, D4, D6
FDIVD	R	243	6	6	4	2	FDIVD D2, D4, D6
FCMPS	R	241	6	8	4	0	FCMPS S4, S6
FCMPD	R	243	6	8	4	0	FCMPD D4, D6
LDURS	D	1506	100	0	4	2	LDURS S2, [X23,100]
LDURD	D	2018	100	0	4	2	LDURD D2, [X23,100]
STURS	D	1504	100	0	4	2	STURS S2, [X23,100]
STURD	D	2016	100	0	4	2	STURD D2, [X23,100]
Field size		11 bits	5 or 9 bits	6 or 2 bits	5 bits	5 bits	All LEGv8 instructions 32 bits

FIGURE 3.17 LEGv8 floating-point architecture revealed thus far. This information is also found in column 2 of the LEGv8 Reference Data Card at the front of this book.

Elaboration: The full ARMv8 instruction set does not use the mnemonics FADDs or FADDd. It just uses FADD, and lets the assembler pick the proper opcode depending if the registers used are S registers and D registers. We worry that it might be confusing to use the same mnemonic for both opcodes, so for teaching purposes LEGv8 distinguishes the two cases with different mnemonics. LEGv8 follows the same decision for the rest of the floating-point arithmetic and data transfer operations.

Hardware/ Software Interface

One issue that architects face in supporting floating-point arithmetic is whether to select the same registers used by the integer instructions or to add a special set for floating point. Because programs normally perform integer operations and floating-point operations on different data, separating the registers will only slightly increase the number of instructions needed to execute a program. The major impact is to create a distinct set of data transfer instructions to move data between floating-point registers and memory.

The benefits of separate floating-point registers are having twice as many registers without using up more bits in the instruction format, having twice the register bandwidth by having separate integer and floating-point register sets, and being able to customize registers to floating point; for example, some computers convert all sized operands in registers into a single internal format.

EXAMPLE

Compiling a Floating-Point C Program into LEGv8 Assembly Code

Let's convert a temperature in Fahrenheit to Celsius:

```
float f2c (float fahr)
{
    return ((5.0/9.0) *(fahr - 32.0));
}
```

Assume that the floating-point argument `fahr` is passed in `S12` and the result should go in `S0`. What is the LEGv8 assembly code?

ANSWER

We assume that the compiler places the three floating-point constants in memory within easy reach of register `X27`. The first two instructions load the constants 5.0 and 9.0 into floating-point registers:

```
f2c:
    LDURs S16, [X27,const5] // S16 = 5.0 (5.0 in memory)
    LDURs S18, [X27,const9] // S18 = 9.0 (9.0 in memory)
```

They are then divided to get the fraction 5.0/9.0:

```
FDIVS S16, S16, S18 // S16 = 5.0 / 9.0
```

(Many compilers would divide 5.0 by 9.0 at compile time and save the single constant 5.0/9.0 in memory, thereby avoiding the divide at runtime.) Next, we load the constant 32.0 and then subtract it from *fahr* (S12):

```
LDURS S18, [X27,const32] // S18 = 32.0
FSUBS S18, S12, S18      // S18 = fahr - 32.0
```

Finally, we multiply the two intermediate results, placing the product in S0 as the return result, and then return

```
FMULS S0, S16, S18 // S0 = (5/9)*(fahr - 32.0)
BR LR              // return
```

Now let's perform floating-point operations on matrices, code commonly found in scientific programs.

Compiling Floating-Point C Procedure with Two-Dimensional Matrices into LEGv8

EXAMPLE

Most floating-point calculations are performed in double precision. Let's perform matrix multiply of $C = C + A * B$. It is commonly called *DGEMM*, for Double precision, General Matrix Multiply. We'll see versions of DGEMM again in Section 3.9 and subsequently in Chapters 4, 5, and 6. Let's assume C, A, and B are all square matrices with 32 elements in each dimension.

```
void mm (double c[][], double a[][], double b[][])
{
    size_t i, j, k;
    for (i = 0; i < 32; i = i + 1)
        for (j = 0; j < 32; j = j + 1)
            for (k = 0; k < 32; k = k + 1)
                c[i][j] = c[i][j] + a[i][k] * b[k][j];
}
```

The array starting addresses are parameters, so they are in X0, X1, and X2. Assume that the integer variables are in X19, X20, and X21, respectively. What is the LEGv8 assembly code for the body of the procedure?

Note that $c[i][j]$ is used in the innermost loop above. Since the loop index is k , the index does not affect $c[i][j]$, so we can avoid loading and storing

ANSWER

$c[i][j]$ each iteration. Instead, the compiler loads $c[i][j]$ into a register outside the loop, accumulates the sum of the products of $a[i][k]$ and $b[k][j]$ in that same register, and then stores the sum into $c[i][j]$ upon termination of the innermost loop. We keep the code simpler by using the assembly language pseudoinstruction `LDI`, which loads a constant into a register.

The body of the procedure starts with saving the loop termination value of 32 in a temporary register and then initializing the three *for* loop variables:

```
mm: ...
    LDI    X10, 32    // X10 = 32 (row size/loop end)
    LDI    X19, 0     // i = 0; initialize 1st for loop
L1:  LDI    X20, 0     // j = 0; restart 2nd for loop
L2:  LDI    X21, 0     // k = 0; restart 3rd for loop
```

To calculate the address of $c[i][j]$, we need to know how a 32×32 , two-dimensional array is stored in memory. As you might expect, its layout is the same as if there were 32 single-dimensional arrays, each with 32 elements. So the first step is to skip over the i “single-dimensional arrays,” or rows, to get the one we want. Thus, we multiply the index in the first dimension by the size of the row, 32. Since 32 is a power of 2, we can use a shift instead:

```
LSL  X11, X19, 5    // X11 = i * 25(size of row of c)
```

Now we add the second index to select the j th element of the desired row:

```
ADD  X11, X11, X20   // X11 = i * size(row) + j
```

To turn this sum into a byte index, we multiply it by the size of a matrix element in bytes. Since each element is 8 bytes for double precision, we can instead shift left by three:

```
LSL  X11, X11, 3     // X11 = byte offset of [i][j]
```

Next we add this sum to the base address of c , giving the address of $c[i][j]$, and then load the double precision number $c[i][j]$ into $D4$:

```
ADD    X11, X0, X11    // X11 = byte address of c[i][j]
LDURD  D4, [X11,#0]    // D4 = 8 bytes of c[i][j]
```

The following five instructions are virtually identical to the last five: calculate the address and then load the double precision number $b[k][j]$.

```
L3: LSL    X9, X21, 5    // X9 = k * 25(size of row of b)
    ADD    X9, X9, X20    // X9 = k * size(row) + j
    LSL    X9, X9, 3     // X9 = byte offset of [k][j]
    ADD    X9, X2, X9     // X9 = byte address of b[k][j]
    LDURD  D16, [X9,#0]  // D16 = 8 bytes of b[k][j]
```

Similarly, the next five instructions are like the last five: calculate the address and then load the double precision number $a[i][k]$.

```
LSL    X9, X19, 5    // X9 = i * 25(size of row of a)
ADD    X9, X9, X21    // X9 = i * size(row) + k
LSL    X9, X9, 3     // X9 = byte offset of [i][k]
ADD    X9, X1, X9     // X9 = byte address of a[i][k]
LDURD  D18, [X9,#0]  // D18 = 8 bytes of a[i][k]
```

Now that we have loaded all the data, we are finally ready to do some floating-point operations! We multiply elements of a and b located in registers D18 and D16, and then accumulate the sum in D4.

```
FMULD  D16, D18, D16 // D16 = a[i][k] * b[k][j]
FADDD  D4, D4, D16   // f4 = c[i][j] + a[i][k] * b[k][j]
```

The final block increments the index k and loops back if the index is not 32. If it is 32, and thus the end of the innermost loop, we need to store the sum accumulated in D4 into $c[i][j]$.

```
ADDI   X21, X21, 1    // $k = k + 1
CMP     X21, X10       // test k vs. 32
B.LT    L3            // if (k < 32) go to L3
STURD   D4, [X11,0]   // = D4
```

Similarly, these final six instructions increment the index variable of the middle and outermost loops, looping back if the index is not 32 and exiting if the index is 32.

```
ADDI   X20, X20, #1    // $j = j + 1
CMP     X20, X10       // test j vs. 32
B.LT    L2            // if (j < 32) go to L2
ADDI   X19, X19, #1    // $i = i + 1
CMP     X19, X10       // test i vs. 32
B.LT    L1            // if (i < 32) go to L1
. . .
```

Looking ahead, [Figure 3.23](#) below shows the x86 assembly language code for a slightly different version of DGEMM in [Figure 3.22](#).

Elaboration: C and many other programming languages use the array layout discussed in the example, called *row-major order*. Fortran instead uses *column-major order*, whereby the array is stored column by column.

Elaboration: Another reason for separate integers and floating-point registers is that microprocessors in the 1980s didn't have enough transistors to put the floating-point unit on the same chip as the integer unit. Hence, the floating-point unit, including the floating-point registers, was optionally available as a second chip. Such optional accelerator chips are called *coprocessor chips*. Since the early 1990s, microprocessors have integrated floating point (and just about everything else) on chip, and thus the term *coprocessor chip* joins *accumulator* and *core memory* as quaint terms that date the speaker.

Elaboration: As mentioned in Section 3.4, accelerating division is more challenging than multiplication. In addition to SRT, another technique to leverage a fast multiplier is *Newton's iteration*, where division is recast as finding the zero of a function to produce the reciprocal $1/c$, which is then multiplied by the other operand. Iteration techniques *cannot* be rounded properly without calculating many extra bits. A TI chip solved this problem by calculating an extra-precise reciprocal.

Elaboration: Java embraces IEEE 754 by name in its definition of Java floating-point data types and operations. Thus, the code in the first example could have well been generated for a class method that converted Fahrenheit to Celsius.

The second example above uses multiple dimensional arrays, which are not explicitly supported in Java. Java allows arrays of arrays, but each array may have its own length, unlike multiple dimensional arrays in C. Like the examples in [Chapter 2](#), a Java version of this second example would require a good deal of checking code for array bounds, including a new length calculation at the end of row accesses. It would also need to check that the object reference is not null.

guard The first of two extra bits kept on the right during intermediate calculations of floating-point numbers; used to improve rounding accuracy.

round Method to make the intermediate floating-point result fit the floating-point format; the goal is typically to find the nearest number that can be represented in the format. It is also the name of the second of two extra bits kept on the right during intermediate floating-point calculations, which improves rounding accuracy.

Accurate Arithmetic

Unlike integers, which can represent exactly every number between the smallest and largest number, floating-point numbers are normally approximations for a number they can't really represent. The reason is that an infinite variety of real numbers exists between, say, 0 and 1, but no more than 2^{53} can be represented exactly in double precision floating point. The best we can do is getting the floating-point representation close to the actual number. Thus, IEEE 754 offers several modes of rounding to let the programmer pick the desired approximation.

Rounding sounds simple enough, but to round accurately requires the hardware to include extra bits in the calculation. In the preceding examples, we were vague on the number of bits that an intermediate representation can occupy, but clearly, if every intermediate result had to be truncated to the exact number of digits, there would be no opportunity to round. IEEE 754, therefore, always keeps two extra bits on the right during intervening additions, called **guard** and **round**, respectively. Let's do a decimal example to illustrate their value.